EXPERIMENT NO. 1

Version Control Operations Using Git

Name: Tejas Mandhyani **Roll Number: 2511135** **Batch: T23**

LO2: Use git effectively for version control, source management and team development

1. Aim

To study and perform basic version control operations using Git, including repository initialization, staging, committing, branching, merging, and pushing code to a remote repository (GitHub).

2. Objectives

- To understand the concept and importance of Version Control Systems (VCS).
- To install and configure Git on a local system.
- To create and manage a local Git repository.
- To perform staging and committing of files.
- To create, switch, and merge branches in Git.
- To connect a local repository to a remote repository on GitHub and push changes.

3. Software Requirements

- Git (latest stable version) installed on the system
- Command Line Interface / Terminal (Git Bash, Command Prompt, or equivalent)
- A GitHub account with internet access
- A text editor (optional, for viewing file contents)

4. Theory

Version Control is the practice of tracking and managing changes to files over time, enabling multiple people to collaborate on a project while maintaining a complete history of modifications. It allows developers to revert to earlier versions, compare changes, and understand who made a particular change and why. Version control systems are broadly classified as centralized or distributed, with Git being the most widely used distributed system today.

Git is a free and open-source distributed version control system created by Linus Torvalds in 2005. Unlike centralized systems, Git gives every developer a complete copy of the project history on their local machine, which allows ofline work, faster operations, and greater resilience against data loss. Git tracks changes at the level of the entire project through a series of snapshots rather than file differences alone.

A repository (commonly called a 'repo') is the central structure in Git that stores all project files along with the complete history of changes made to them. A repository can exist locally on a developer's machine or remotely on a hosting platform. It contains a hidden .git directory that holds all metadata, configuration, and object history required for version tracking.

A commit is a snapshot of the project at a specific point in time. Each commit is uniquely identified by a hash and includes information such as the author, timestamp, and a descriptive message explaining the change. Commits form the building blocks of a project's history and allow developers to track the evolution of the codebase step by step.

A branch is an independent line of development within a repository. By default, Git creates a branch named

'master' or 'main'. Branches allow developers to work on new features, experiments, or bug fixes in isolation without affecting the stable codebase. Once the work on a branch is complete and tested, it can be integrated back into the main branch.

Merging is the process of combining changes from one branch into another. When a merge is performed, Git attempts to automatically integrate the changes; if the same lines of code have been modified differently in both branches, a merge conflict occurs, which must be resolved manually.

GitHub is a cloud-based hosting service for Git repositories that provides additional collaboration features such as pull requests, issue tracking, code review, and project management tools. By connecting a local Git repository to a remote repository on GitHub, developers can back up their work, collaborate with others, and maintain a publicly or privately accessible version of their project. Commands such as git remote and git push are used to establish and utilize this connection between local and remote repositories.

5. Commands Used

Command	Purpose
`git --version`	Check installed Git version
`mkdir GitExperiment`	Create a new directory for the project
`cd GitExperiment`	Navigate into the directory
`pwd`	Print current working directory
`ls`	List files and folders in current directory
`git config --global user.name "Your Name"`	Set global username for Git commits
`git config --global user.email "your@email.com"`	Set global email for Git commits
`git config --list`	Display all Git configuration settings
`git init`	Initialize a new Git repository
`git status`	Show current state of working directory and staging area
`echo Hello Git > file1.txt`	Create a text file with sample content
`cat file1.txt`	Display contents of a file
`git add file1.txt`	Stage a specific file for commit
`git commit -m "message"`	Record staged changes to repository history
`git log`	View commit history in detail
`git add .`	Stage all modified/new files in the directory
`git log --oneline`	View condensed, one-line-per-commit history

`git branch <name>`	Create a new branch
`git branch`	List all local branches
`git checkout <branch>`	Switch to the specified branch
`git checkout master`	Switch back to the master branch
`git merge <branch>`	Merge specified branch into current branch
`git log --oneline --graph`	View commit history as a graph
`git remote add origin <url>`	Link local repository to a remote GitHub repository
Command	**Purpose**
`git remote -v`	View configured remote repository URLs
`git branch -M main`	Rename current branch to main
`git push -u origin main`	Push local commits to remote repository and set upstream

6. Procedure

Step 1

```
git --version
```

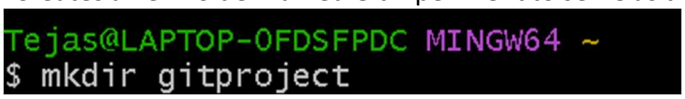
Verifies that Git is installed on the system and displays its version number.



Step 2

```
mkdir GitExperiment
```

Creates a new folder named GitExperiment to serve as the project directory.



Step 3

```
cd GitExperiment
```

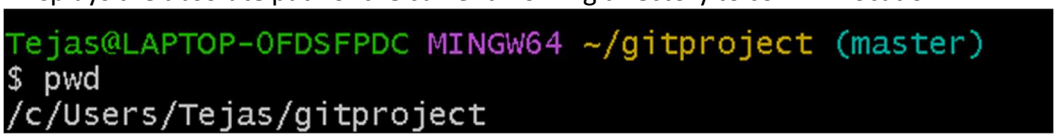
Changes the current directory into the newly created GitExperiment folder.



Step 4

```
pwd
```

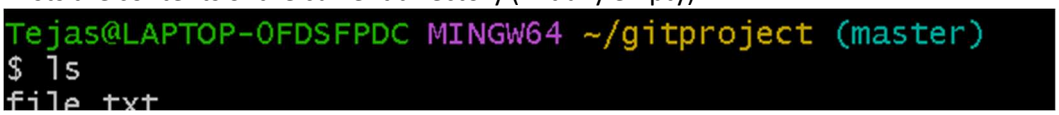
Displays the absolute path of the current working directory to confirm location.



Step 5

```
ls
```

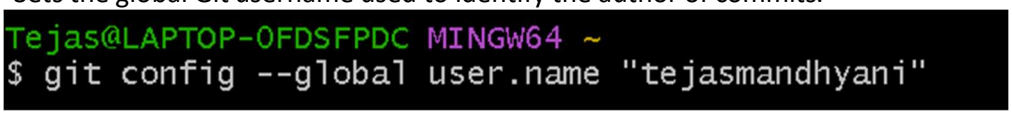
Lists the contents of the current directory (initially empty).



Step 6

```
git config --global user.name "Your Name"
```

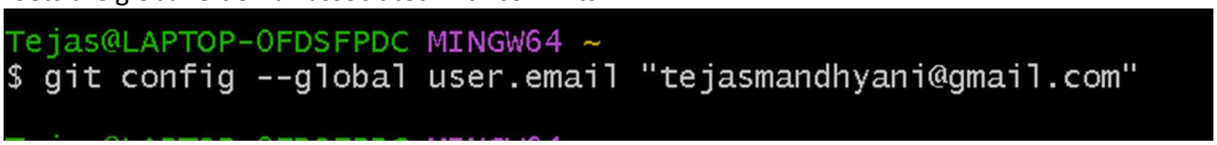
Sets the global Git username used to identify the author of commits.



Step 7

```
git config --global user.email "your@email.com"
```

Sets the global Git email associated with commits.



Step 8

```
git config --list
```

Displays the complete list of configured Git settings to verify the username and email.

```
Tejas@LAPTOP-0FDSFPDC MINGW64 ~
$ git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=openssl
http.sslcainfo=C:/Program Files/Git/mingw64/etc/ssl/certs/ca-bundle.crt
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=master
user.name=tejasmandhyani
user.email=tejasmandhyani@gmail.com
```

Step 9

`git init`

Initializes an empty Git repository in the current directory, creating a hidden .git folder.

```
Tejas@LAPTOP-0FDSFPDC MINGW64 ~/gitproject
$ git init
Initialized empty Git repository in C:/Users/Tejas/gitproject/.git/
```

Step 10

`git status`

Displays the current status of the repository, confirming it is empty and clean.

```
Tejas@LAPTOP-0FDSFPDC MINGW64 ~/gitproject (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        file.txt

nothing added to commit but untracked files present (use "git add" to track)
```

Step 11

```
echo Hello Git > file1.txt
```

Creates a new file, file1.txt, containing the text "Hello Git".

```
Tejas@LAPTOP-OFDSFPDC MINGW64 ~/gitproject (master)
$ echo "Hello Git" > file.txt
```

Step 12

```
cat file1.txt
```

Displays the contents of file1.txt to confirm it was created correctly.

```
Tejas@LAPTOP-OFDSFPDC MINGW64 ~/gitproject (master)
$ cat file.txt
Hello Git
```

Step 13

```
git add file1.txt
```

Stages file1.txt, moving it to the staging area in preparation for a commit.

```
Tejas@LAPTOP-OFDSFPDC MINGW64 ~/gitproject (master)
$ git add file.txt
warning: in the working copy of 'file.txt', LF will be replaced by CRLF the next time Git touches it
```

Step 14

```
git status
```

Confirms that file1.txt is now staged and ready to be committed.

```
Tejas@LAPTOP-OFDSFPDC MINGW64 ~/gitproject (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        file.txt

nothing added to commit but untracked files present (use "git add" to track)
```

Step 15

```
git commit -m "First Commit"
```

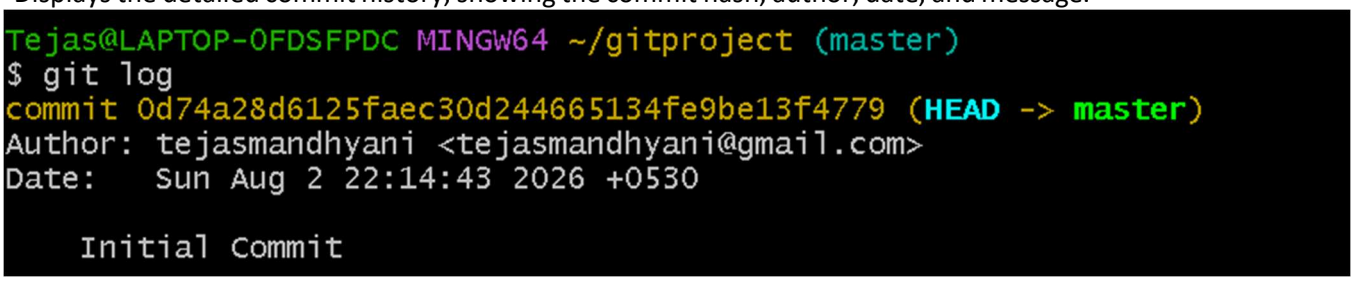
Commits the staged file to the repository with the message "First Commit".

```
Tejas@LAPTOP-OFDSFPDC MINGW64 ~/gitproject (master)
$ git commit -m "Initial Commit"
[master (root-commit) 0d74a28] Initial Commit
 1 file changed, 1 insertion(+)
 create mode 100644 file.txt
```

Step 16

```
git log
```

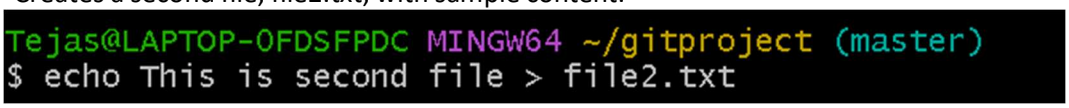
Displays the detailed commit history, showing the commit hash, author, date, and message.



Step 17

```
echo This is second file > file2.txt
```

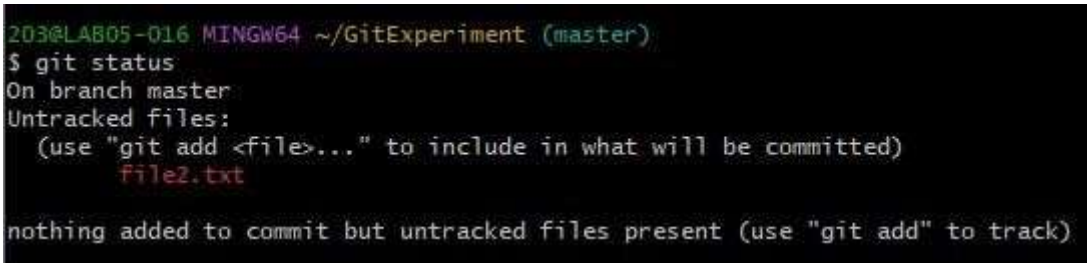
Creates a second file, file2.txt, with sample content.



Step 18

```
git status
```

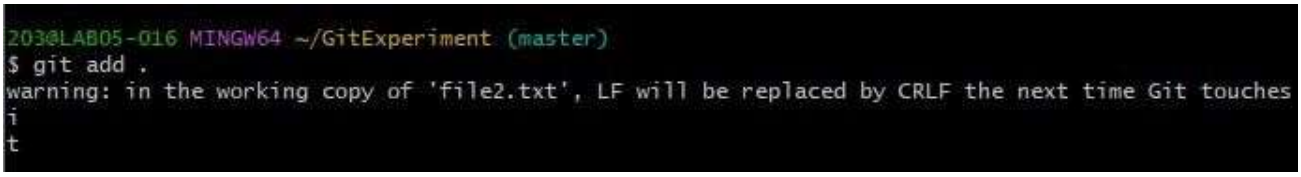
Shows that file2.txt is untracked and not yet staged.



Step 19

```
git add .
```

Stages all changes in the working directory, including the new file2.txt.



Step 20

```
git status
```

Confirms that file2.txt is now staged for commit.

```
Tejas@LAPTOP-OFDSFPDC MINGW64 ~/gitproject (master)
$ git status
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        file2.txt

nothing added to commit but untracked files present (use "git add" to track)
```

Step 21

```
git commit -m "Added second file"
```

Commits the staged changes with the message "Added second file".

```
Tejas@LAPTOP-OFDSFPDC MINGW64 ~/gitproject (master)
$ git commit -m "Added Second File"
On branch master
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        file2.txt

nothing added to commit but untracked files present (use "git add" to track)
```

Step 22

```
git log --oneline
```

Displays a condensed commit history showing both commits made so far.

```
Tejas@LAPTOP-OFDSFPDC MINGW64 ~/gitproject (master)
$ git log --oneline
0d74a28 (HEAD -> master) Initial Commit
```

Step 23

```
git branch newbranch
```

Creates a new branch named newbranch from the current commit.

```
Tejas@LAPTOP-OFDSFPDC MINGW64 ~/gitproject (master)
$ git branch newbranch
```

Step 24

```
git branch
```

Lists all branches in the repository and highlights the currently active branch.

```
Tejas@LAPTOP-OFDSFPDC MINGW64 ~/gitproject (master)
$ git branch
* master
  newbranch
```

Step 25

```
git checkout newbranch
```

Switch the working directory to newbranch.

```
Tejas@LAPTOP-OFDSFPDC MINGW64 ~/gitproject (master)
$ git checkout newbranch
Switched to branch 'newbranch'
```

Step 26

```
echo Branch File > branch.txt
```

Creates a new file, branch.txt, specific to the new branch.

```
Tejas@LAPTOP-OFDSFPDC MINGW64 ~/gitproject (newbranch)
$ echo Branch File > branch.txt
```

Step 27

`git add .`

Stages the newly created branch.txt file.

```
Tejas@LAPTOP-OFDSFPDC MINGW64 ~/gitproject (newbranch)
$ git add .
warning: in the working copy of 'branch.txt', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'file2.txt', LF will be replaced by CRLF the next time Git touches it
```

Step 28

`git commit -m "Added branch file"`

Commits branch.txt on newbranch with an appropriate message.

```
Tejas@LAPTOP-OFDSFPDC MINGW64 ~/gitproject (newbranch)
$ git commit -m "Added Branch File"
[newbranch b1eb496] Added Branch File
 2 files changed, 2 insertions(+)
 create mode 100644 branch.txt
 create mode 100644 file2.txt
```

Step 29

`git checkout master`

Switches back to the master branch.

```
Tejas@LAPTOP-OFDSFPDC MINGW64 ~/gitproject (newbranch)
$ git checkout master
Switched to branch 'master'
```

Step 30

`git merge newbranch`

Merges the changes from newbranch into the master branch.

```
Tejas@LAPTOP-OFDSFPDC MINGW64 ~/gitproject (master)
$ git merge newbranch
Updating 0d74a28..b1eb496
Fast-forward
 branch.txt | 1 +
 file2.txt  | 1 +
 2 files changed, 2 insertions(+)
 create mode 100644 branch.txt
 create mode 100644 file2.txt
```

Step 31

`git log --oneline --graph`

Displays the commit history as a graph, showing the branch and merge structure.

```
Tejas@LAPTOP-OFDSFPDC MINGW64 ~/gitproject (master)
$ git log --oneline --graph
* b1eb496 (HEAD -> master, newbranch) Added Branch File
* 0d74a28 Initial Commit
```

Step 32

`git remote add origin` https://github.com/tejasmandhyani/MyFirstRepo

Links the local repository to a remote repository hosted on GitHub, named origin.

```
Tejas@LAPTOP-OFDSFPDC MINGW64 ~/gitproject (master)
$ git remote add origin https://github.com/tejasmandhyani/MyFirstRepo
```

Step 33

```
git remote -v
```

Verifies the configured remote repository URLs for fetch and push operations.

```
Tejas@LAPTOP-OFDSFPDC MINGW64 ~/gitproject (master)
$ git remote -v
origin  https://github.com/tejasmandhyani/MyFirstRepo (fetch)
origin  https://github.com/tejasmandhyani/MyFirstRepo (push)
```

Step 34

```
git branch -M main
```

Renames the current branch from master to main, following GitHub's default naming convention.

```
Tejas@LAPTOP-OFDSFPDC MINGW64 ~/gitproject (master)
$ git branch -M main
```

7. Observation

On executing the above commands, Git successfully tracked all changes made within the GitExperiment directory. The git status command correctly reflected the state of files as untracked, staged, or committed at each stage. The git log and git log --oneline commands displayed an accurate, chronological history of all commits. A new branch (newbranch) was successfully created and merged into the master branch without conflicts, since the changes involved different, non-overlapping files. The repository was successfully linked to a remote GitHub repository, and the branch was renamed to main to match GitHub's default convention prior to pushing.

8. Result

The version control operations using Git including repository initialization, staging, committing, branching, merging, and pushing to a remote GitHub repository were performed successfully, and the expected outcomes were observed at each stage.

9. Conclusion

Through this experiment, the fundamental concepts and operations of Git as a distributed version control system were studied and practically implemented. The exercise provided hands-on experience in initializing repositories, tracking file changes, creating commits, managing branches, merging code, and synchronizing a local repository with a remote GitHub repository. This forms the foundation for collaborative software development practices used extensively in the industry.